
A)

Write a method that increases the value of every node between positions **i** and **j** (inclusive) by **+10**.

Assume the list has at least **j** nodes.

(8 points)

Solution:

```
public void increaseRange(int i, int j) {
```

```
}
```

B)

Write a **recursive** method that returns true if the array **num[]** is sorted in ascending order, and false otherwise.

(7 points)

Example:

isSorted({1,2,3,4}, size) → true

isSorted({2,1,3}, size) → false

Solution:

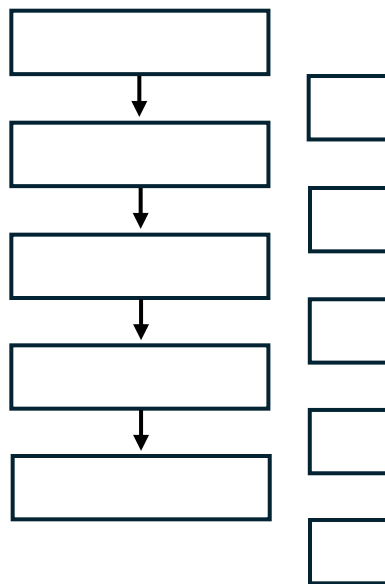
```
public static boolean isSorted(int[] num, int size) {
```

```
}
```

C)

Consider the recursive method shown below. For the function call **fun(10, 4)**, draw a complete recursion trace that includes every recursive call and its corresponding return value. **(10 points)**

```
public static int fun(int a, int b) {
    if (a <= 0 || b <= 0) {
        return a + b;
    }
    int r = a % b;
    if (r == 0) {
        return a + b + fun(a / 2, b - 1);
    }
    if (a > b) {
        return r + fun(a - 2, b - 1);
    } else {
        return b + fun(a - 1, r);
    }
}
```

Solution:

Determine the **Big-O running time** of the following code fragments, in terms of **n**.

(Each 3 points – last 4 points)

Code	Running Time
<pre> int sum = 0; for (int i = 0; i < n; i++) { for (int j = 0; j <= i; j++) { sum += j; } for (int k = 0; k < 50; k++) { sum++; } } </pre>	
<pre> int count = 0; for (int i = 1; i <= n; i++) { int x = i; while (x < n) { count++; x = x * 2; } } </pre>	
<pre> int total = 0; for (int i = 1; i <= n; i++) { for (int j = 0; j < n; j++) { int t = i + j; while (t > 0) { total += t; t = t / 2; } } } </pre>	

A)

Evaluate the following **postfix expression** using a stack. Additionally, you must show the contents of the stack at the indicated points (**A, B, and C**) in the postfix expression. Also, give the final result. **(10 points)**

$$2 \quad 13 \quad \overset{A}{+} \quad 5 \quad - \quad 6 \quad 3 \quad \overset{B}{/} \quad 5 \quad \overset{C}{*} \quad +$$

A

B

C

Final Result: _____

B)

Consider the following method, assume the standard **Stack** operations (**push**, **pop**, **top**, **isEmpty**) and **Queue** operations (**enqueue**, **dequeue**, **isEmpty**, **size**) are available.

You are given the following initial values:

(7 points)

- **Stack S (top → bottom):** Top → {8, 3, 1, 5}
 - **Queue Q (front → back):** {} (empty)
-

```
public static void fun(Stack<Integer> S, Queue<Integer> Q) {  
    // Step 1  
    while (!S.isEmpty()) {  
        Q.enqueue(S.pop());  
    }  
    // Step 2  
    int k = Q.size() / 2;  
    for (int i = 0; i < k; i++) {  
        Q.enqueue(Q.dequeue());  
    }  
    // Step 3  
    while (!Q.isEmpty()) {  
        S.push(Q.dequeue());  
    }  
}
```

Trace the execution of fun() and answer the following:

1. The content of the stack and the queue after the first step:

Stack:

Queue:

2. The content of the queue after the second step:

Queue:

3. The content of the stack and the queue after the third step:

Stack:

Queue:

C) Choose the correct answer for each of the following questions
(points)

(8

1. A circular queue has **maxSize = 6** and **front = 5**, **rear = 1**, and **count = 4**.
A dequeue operation is performed, what will the new value of **front**?

- a) 5
- b) 0
- c) 1
- d) 6

2. Which real-world situation is best modeled using a **queue**, not a stack?

- a) Undo operations in a text editor
- b) Evaluating a postfix expression
- c) Order of print requests sent to a printer
- d) Recursive function call execution

3. When implementing a queue using a **singly linked list**, which statement is **TRUE**?

- a) Enqueue requires traversal of the entire list
- b) Dequeue is performed at the rear
- c) Enqueue is $O(1)$ when the queue stores a tail pointer
- d) Dequeue must update both front and rear pointers every time

4. Which of the following statements is **TRUE**?

- a) A queue removes the most recently inserted element
- b) A stack and queue both support FIFO removal
- c) Moving all items from a queue to a stack reverses their order
- d) Moving all items from a stack to a queue reverses their order

Question No. 4 (Tree)**(SO 2 – CLO 11 – 20 Marks)**

Insert the following keys into an initially empty **Binary Search Tree (BST)**.

Insert them in the given order from left to right.

Answer parts **(B)**, **(C)**, **(D)**, and **(E)** based on the resulting tree.

Keys: 50, 25, 75, 10, 30, 60, 90, 5, 15, 28, 55, 85

A) Draw the **Binary Search Tree** after inserting all keys in the given order.

(6 Marks)

B) Write the **Post-Order traversal** of the tree

(3 Marks)

Post-Order:

C) For each statement, circle **True** or **False**

(3 Marks)

Statement	True	False
1. This BST is a Complete Binary Tree	True	False
2. This BST is a Full Binary Tree	True	False
3. The height of this BST is 3	True	False

D) Delete node **25** from the tree. Draw the resulting tree after deletion.

(3 Marks)

E) Write a **recursive** method that counts the number of nodes containing **even numbers** in a BST.

(5 Marks)

```
public int countEven(Node root)
{
```

```
}
```

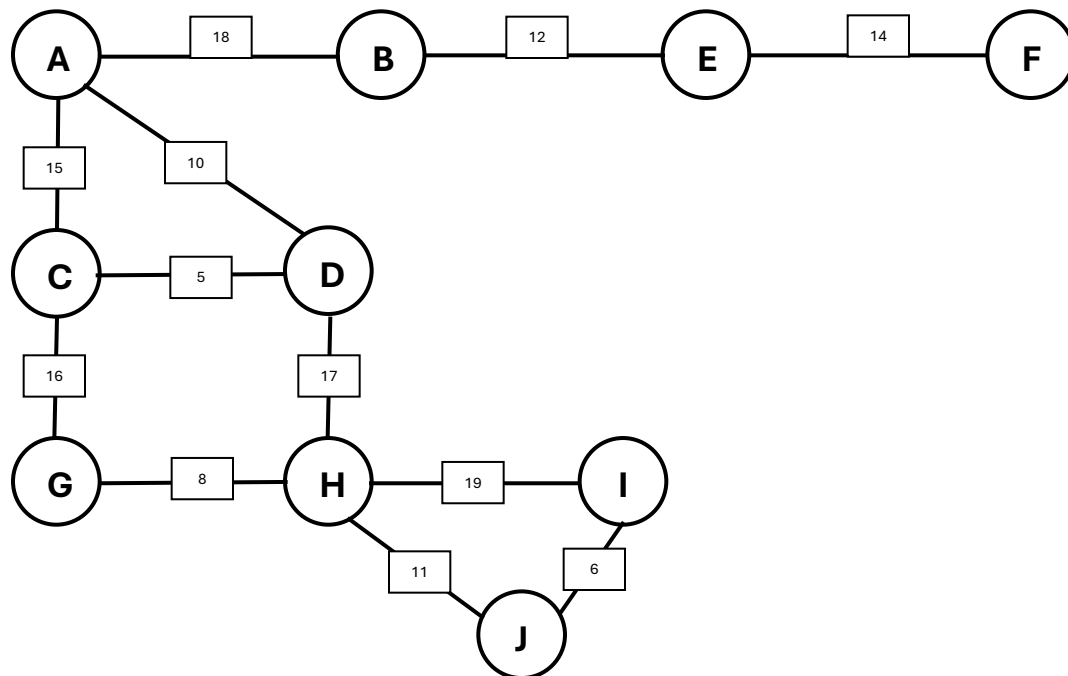

A city has **10 pharmacies (A, B, C, D, E, F, G, H, I, J)** connected by roads.

Each road has a congestion level (weight).

Ali starts at **Pharmacy A** and wants to visit all pharmacies using **Depth-First Search (DFS)**.

Write the order in which he visits all pharmacies.

Rule: Visit the pharmacy connected by the road with the **HIGHEST** congestion level first.



Answer :

_____ → _____ → _____ → _____ → _____ → _____ → _____ → _____ → _____ → _____

Question No. 6**(SO 1, CLO 15 – 10 Marks)**

You have the following keys that need to be stored for fast retrieval.
Use **Double Hashing** to insert them into a hash table.

Given:	Table Size m = 11 $h_1(k) = k \bmod 11$ $h_2(k) = 7 - (k \bmod 7)$
Keys to Insert:	23, 45, 67, 12, 34, 56, 78, 89

Hash Table:

Index	0	1	2	3	4	5	6	7	8	9	10
Value											